

★ RELATIONAL DATABASES ★

★ Project Title : - ✨ Product Dissection For Rapido ✨

★ Project Type :- ✨ Database Schema Design And Implementation In Real World ✨

★ Project Submitted By : - ✨ Nishant Shaileshkumar Trivedi ✨

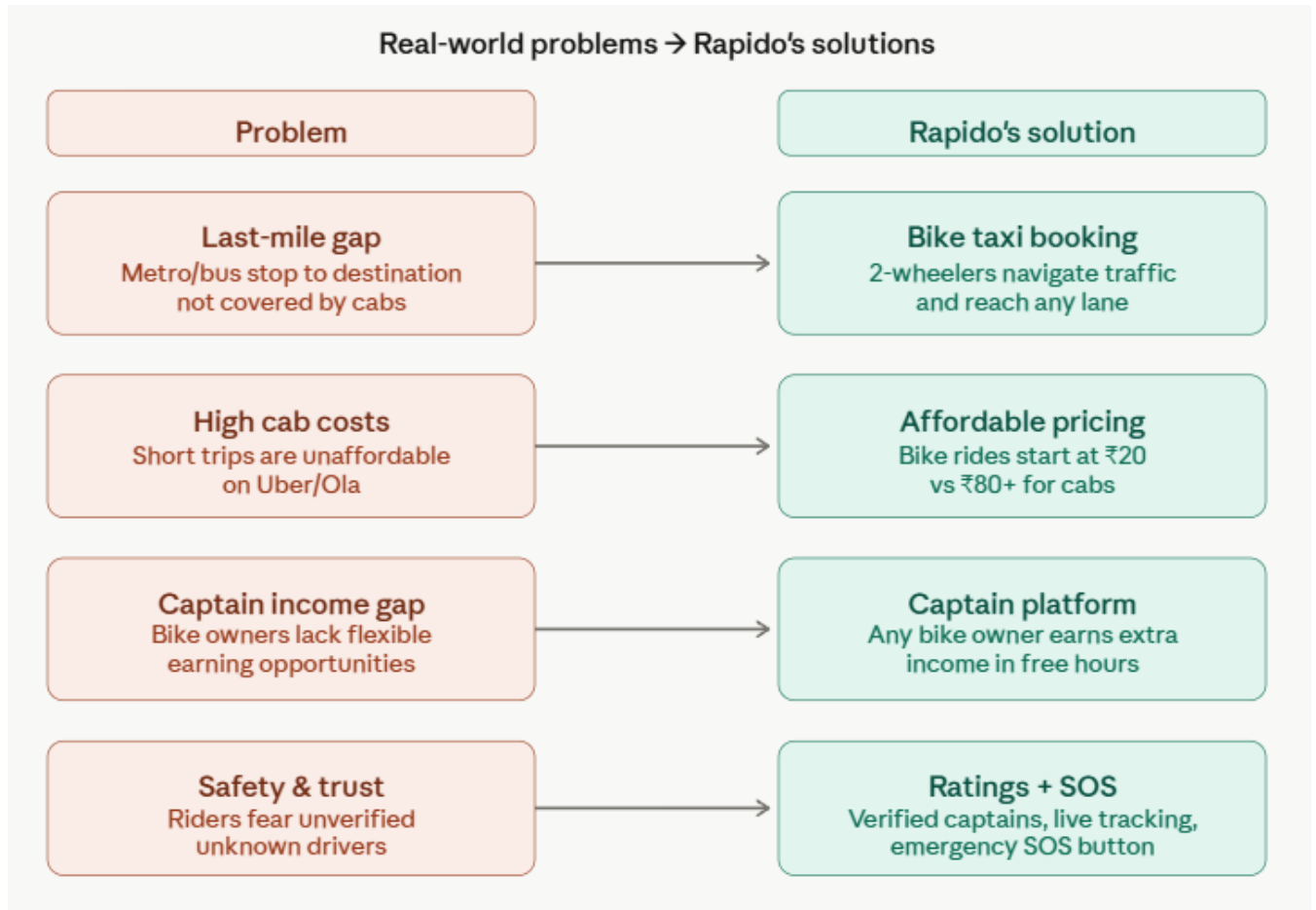


Product Dissection For Rapido

Company Overview

Rapido was founded in 2015 by Aravind Sanka, Pavan Guntupalli, and SR Rishikesh, and is headquartered in Bengaluru, India. It is India's largest bike taxi aggregator, operating in 100+ cities. Rapido connects riders with captains (drivers on bikes and auto-rickshaws) for fast, affordable, last-mile urban travel. It solves the specific Indian urban problem of traffic congestion and expensive cabs by using two-wheelers – far more nimble and cheaper to operate.

Product Dissection and Real-World Problems Solved by Rapido



Case Study: Real-World Problems & Rapido's Approach

Rapido, one of the leading transportation service provider platforms, has not only revolutionised the way we share and consume content but has also addressed significant real-world challenges through its innovative features. By providing a user proper facility for his needs with the competitive and budget friendly options with the help of technology, Rapido has positioned itself as a solution-driven and options friendly platform with safety and security of the users' transportation journey.

Problem 1 – The Last-Mile Gap

❖ Real-World Challenge :

In Indian cities, public transport covers main corridors but fails at the "last mile." A commuter getting off the metro 3 km from their office has no affordable option.

→ Rapido's Solution :

Rapido's bike taxi reaches narrow lanes, bylanes, and gated communities that auto-rickshaws and cabs can't economically serve, solving this gap instantly.

Problem 2 – Prohibitive Short-Trip Cab Costs

❖ Real-World Challenge :

Uber and Ola have minimum fares that make a 2 km trip cost ₹80-120.

→ Rapido's Solution :

Rapido disrupts this by using bikes with lower operating costs, offering the same trip for ₹20-35. This democratizes mobility for daily-wage workers, students, and office commuters.

Problem 3 – Gig Income for Two-Wheeler Owners

❖ Real-World Challenge :

Millions of Indians own bikes but have no structured way to monetize them.

→ Rapido's Solution :

Rapido's Captain model lets anyone with a two-wheeler earn ₹500-1000/day in their own schedule, creating a massive gig economy layer.

Problem 4 – Safety and Trust

❖ Real-World Challenge :

In many cities, riders or drivers are always unverified or unknown. So the trust factor affects the safety and trust purpose.

→ Rapido's Solution :

Rapido addresses trust through Aadhaar-verified captains, profile photos, real-time GPS tracking, and an in-app SOS button – building the confidence especially for women and students who are a key rider demographic.

Conclusion For The Real-World Problems

Rapido proves itself as a best and reliable transportation service provider option which helps the user or a customer to choose convenient options for more reliability with competitive fares based on his/her budget with safety and security. With a completion of the last mile gap and short ride fare cost helps the customer to save unnecessary pay in comparison to the other options. With the consideration of extra or passive income sources, Rapido proves itself as a different and unique approach for gig workers. Within a part time, Rapido Riders could earn a handsome passive income also. All of these reasons make Rapido a trustworthy and reliable option for both users and riders.

Top Features Of Rapido

Rapido's top features are:

- **Ride Booking —**

Users book bike rides, auto rides, or parcels in real time

- **Captain (Driver) System —**

Captains register, get verified, and accept ride requests

- **Real-Time Tracking —**

GPS-based live tracking of captain to user

- **Dynamic Pricing —**

Surge pricing based on demand/supply in an area

- **Wallet & Payments —**

In-app wallet, UPI, cash support

- **Ratings & Reviews —**

Mutual rating between users and captains

- **Ride History —**

Past rides stored per user and captain

- **Promo & Offers —**

Coupon codes and referral rewards

Schema Design Based on Top Features

Now here is the core SQL/database work. Here are all the entities, their attributes, and relationships.

Entity 1: **Users**

Attribute	Type	Description
UserID (PK)	INT	Unique user identifier
Name	VARCHAR(100)	Full name of rider
Phone	VARCHAR(15)	Mobile number (used for login)
Email	VARCHAR(100)	Email address
City	VARCHAR(50)	User's home city
Registration_Date	DATE	When the user signed up
Wallet_Balance	DECIMAL(10,2)	Current wallet amount

Entity 2: **Captains** (Drivers)

Attribute	Type	Description
CaptainID (PK)	INT	Unique captain identifier
Name	VARCHAR(100)	Captain's full name
Phone	VARCHAR(15)	Contact number
Aadhaar_Verified	BOOLEAN	KYC verification status
Vehicle_Type	ENUM	'Bike', 'Auto', 'Parcel'
Vehicle_Number	VARCHAR(20)	Registration plate
City	VARCHAR(50)	Operating city
Rating	DECIMAL(3,2)	Average rating (0-5)
Status	ENUM	'Active', 'Inactive', 'Suspended'

Entity 3: Rides

Attribute	Type	Description
RideID (PK)	INT	Unique ride identifier
UserID (FK)	INT	Rider who booked
CaptainID (FK)	INT	Captain assigned
Pickup_Location	VARCHAR(200)	Start location
Drop_Location	VARCHAR(200)	End location
Pickup_Lat	DECIMAL(9,6)	GPS latitude of pickup
Pickup_Long	DECIMAL(9,6)	GPS longitude of pickup
Drop_Lat	DECIMAL(9,6)	GPS latitude of drop
Drop_Long	DECIMAL(9,6)	GPS longitude of drop
Ride_Date	DATETIME	When ride started
Ride_Type	ENUM	'Bike', 'Auto', 'Parcel'
Distance_KM	DECIMAL(5,2)	Distance covered
Fare	DECIMAL(8,2)	Final fare charged
Status	ENUM	'Requested', 'Accepted', 'Ongoing', 'Completed', 'Cancelled'

Entity 4: **Payments**

Attribute	Type	Description
PaymentID (PK)	INT	Unique payment ID
RideID (FK)	INT	Linked ride
UserID (FK)	INT	Who paid
Amount	DECIMAL(8,2)	Amount paid
Payment_Mode	ENUM	'Cash', 'Wallet', 'UPI', 'Card'
Payment_Date	DATETIME	Time of payment
Status	ENUM	'Success', 'Pending', 'Failed'

Entity 5: Ratings

Attribute	Type	Description
RatingID (PK)	INT	Unique rating ID
RideID (FK)	INT	Ride being rated
Rated_By	ENUM	'User', 'Captain'
Rating_Score	DECIMAL(2,1)	Score out of 5
Review_Text	TEXT	Optional text feedback
Rating_Date	DATETIME	When rating was given

Entity 6: Promos

Attribute	Type	Description
PromoID (PK)	INT	Unique promo ID
Code	VARCHAR(20)	Promo code string
Discount_Type	ENUM	'Flat', 'Percentage'
Discount_Value	DECIMAL(6,2)	Discount amount
Min_Ride_Amount	DECIMAL(6,2)	Minimum fare to apply
Expiry_Date	DATE	When promo expires
Max_Uses	INT	Maximum redemptions allowed

Entity 7: **Promo_Usage**

Attribute	Type	Description
UsageID (PK)	INT	Unique usage record
PromoID (FK)	INT	Promo applied
UserID (FK)	INT	User who used it
RideID (FK)	INT	Ride it was applied to
Used_On	DATETIME	Timestamp of usage

Schema Description

The schema for Rapido involves multiple entities that represent different aspects of the platform. These entities include Users, Captains, Rides, Payments, Ratings, Promo_Usage and more. Each entity has specific attributes that describe its properties and relationships with other entities.

❖ Users and Captains are separate entities

Because they have completely different attributes (Aadhaar verification, vehicle info are captain-specific; wallet balance is user-specific). This avoids a messy combined "person" table.

❖ Rides is the central entity

It connects Users, Captains, Payments, and Ratings. Everything meaningful happens in a ride context, so it holds GPS coordinates, fare, status, and type.

❖ Payments is separated from Rides

Because a single ride could theoretically have partial payments or failed attempts. The one-to-one-or-many design gives flexibility.

❖ Ratings are linked to Rides, not Users directly

Because a rating is always in the context of a specific ride experience — you don't rate a captain in general, you rate the ride you shared.

❖ Promo_Usage is a junction table

Between Promos and Users, capturing the many-to-many relationship (one promo can be used by many users; one user can use many promos over time).

Relationships Summary

- **Users → Rides:**

One user books many rides (1:N)

- **Captains → Rides:**

One captain completes many rides (1:N)

- **Rides → Payments:**

Each ride has one payment record (1:1)

- **Rides → Ratings:**

Each ride can receive ratings from both user and captain
(1:N)

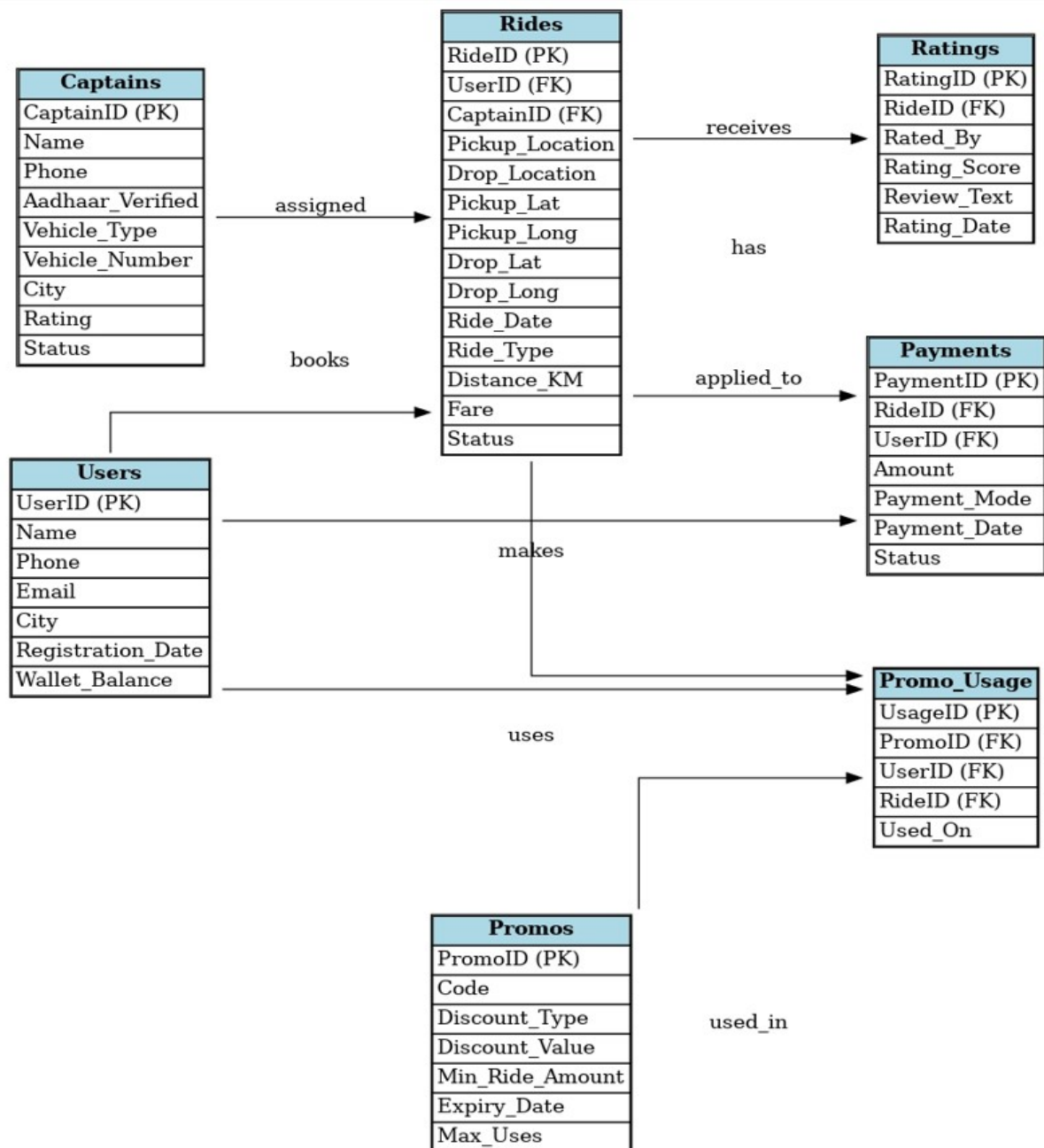
- **Promos ↔ Users:**

Many-to-many via **Promo_Usage** junction table, also linked to the specific **RIDE** it was used on

RIDES is the **central hub** of the entire schema — 5 out of 6 relationship lines pass through it.

ER Diagram

Let's construct an ER diagram that vividly portrays the relationships and attributes of the entities within the Rapido schema. This ER diagram will serve as a visual representation, shedding light on the pivotal components of Rapido's data model. By employing this diagram, you'll gain a clearer grasp of the intricate interactions and connections that define the platform's dynamics.



SQL – CREATE TABLE Statements

(for actual implementation)

Now let's write the real SQL that brings this schema to life:

❖ Table 1: Users

```
CREATE TABLE Users (  
  UserID      INT PRIMARY KEY AUTO_INCREMENT,  
  Name        VARCHAR(100) NOT NULL,  
  Phone       VARCHAR(15) UNIQUE NOT NULL,  
  Email       VARCHAR(100),  
  City        VARCHAR(50),  
  Registration_Date DATE DEFAULT CURRENT_DATE,  
  Wallet_Balance DECIMAL(10,2) DEFAULT 0.00  
);
```

❖ Table 2: Captains

```
CREATE TABLE Captains (  
  CaptainID   INT PRIMARY KEY AUTO_INCREMENT,  
  Name        VARCHAR(100) NOT NULL,  
  Phone       VARCHAR(15) UNIQUE NOT NULL,  
  Aadhaar_Verified BOOLEAN DEFAULT FALSE,  
  Vehicle_Type ENUM('Bike','Auto','Parcel') NOT NULL,  
  Vehicle_Number VARCHAR(20) NOT NULL,  
  City        VARCHAR(50),  
  Rating      DECIMAL(3,2) DEFAULT 5.00,  
  Status      ENUM('Active','Inactive','Suspended') DEFAULT 'Active'  
);
```

❖ Table 3: Rides (central entity)

```
CREATE TABLE Rides (  
  RideID          INT PRIMARY KEY AUTO_INCREMENT,  
  UserID          INT NOT NULL,  
  CaptainID      INT,  
  Pickup_Location VARCHAR(200),  
  Drop_Location  VARCHAR(200),  
  Pickup_Lat     DECIMAL(9,6),  
  Pickup_Long    DECIMAL(9,6),  
  Drop_Lat       DECIMAL(9,6),  
  Drop_Long      DECIMAL(9,6),  
  Ride_Date      DATETIME DEFAULT CURRENT_TIMESTAMP,  
  Ride_Type      ENUM('Bike','Auto','Parcel') NOT NULL,  
  Distance_KM    DECIMAL(5,2),  
  Fare           DECIMAL(8,2),  
  Status  
  ENUM('Requested','Accepted','Ongoing','Completed','Cancelled')  
  DEFAULT 'Requested',  
  FOREIGN KEY (UserID) REFERENCES Users(UserID),  
  FOREIGN KEY (CaptainID) REFERENCES Captains(CaptainID)  
);
```

❖ Table 4: Payments

```
CREATE TABLE Payments (  
  PaymentID      INT PRIMARY KEY AUTO_INCREMENT,  
  RideID         INT NOT NULL,  
  UserID         INT NOT NULL,  
  Amount         DECIMAL(8,2) NOT NULL,  
  Payment_Mode   ENUM('Cash','Wallet','UPI','Card') NOT NULL,  
  Payment_Date   DATETIME DEFAULT CURRENT_TIMESTAMP,  
  Status         ENUM('Success','Pending','Failed') DEFAULT 'Pending',  
  FOREIGN KEY (RideID) REFERENCES Rides(RideID),  
  FOREIGN KEY (UserID) REFERENCES Users(UserID)
```

);

❖ Table 5: Ratings

```
CREATE TABLE Ratings (  
    RatingID    INT PRIMARY KEY AUTO_INCREMENT,  
    RideID      INT NOT NULL,  
    Rated_By    ENUM('User','Captain') NOT NULL,  
    Rating_Score DECIMAL(2,1) CHECK (Rating_Score BETWEEN 1.0  
AND 5.0),  
    Review_Text TEXT,  
    Rating_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (RideID) REFERENCES Rides(RideID)  
);
```

❖ Table 6: Promos

```
CREATE TABLE Promos (  
    PromoID      INT PRIMARY KEY AUTO_INCREMENT,  
    Code          VARCHAR(20) UNIQUE NOT NULL,  
    Discount_Type ENUM('Flat','Percentage') NOT NULL,  
    Discount_Value DECIMAL(6,2) NOT NULL,  
    Min_Ride_Amount DECIMAL(6,2) DEFAULT 0,  
    Expiry_Date   DATE,  
    Max_Uses      INT DEFAULT 1000  
);
```

❖ Table 7: Promo_Usage (junction table)

```
CREATE TABLE Promo_Usage (  
    UsageID INT PRIMARY KEY AUTO_INCREMENT,  
    PromoID INT NOT NULL,  
    UserID INT NOT NULL,  
    RideID INT NOT NULL,  
    Used_On DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (PromoID) REFERENCES Promos(PromoID),  
    FOREIGN KEY (UserID) REFERENCES Users(UserID),  
    FOREIGN KEY (RideID) REFERENCES Rides(RideID)  
);
```

Sample SQL Queries

(to showcase how data is used)

→ Q1: Total rides completed by a specific captain

```
SELECT c.Name, COUNT(r.RideID) AS Total_Rides  
FROM Captains c  
JOIN Rides r ON c.CaptainID = r.CaptainID  
WHERE r.Status = 'Completed'  
GROUP BY c.CaptainID;
```

→ Q2: Average rating of each captain

```
SELECT c.Name, AVG(rt.Rating_Score) AS Avg_Rating  
FROM Captains c  
JOIN Rides r ON c.CaptainID = r.CaptainID  
JOIN Ratings rt ON r.RideID = rt.RideID  
WHERE rt.Rated_By = 'User'  
GROUP BY c.CaptainID  
ORDER BY Avg_Rating DESC;
```

→ Q3: Total revenue per city (completed rides only)

```
SELECT u.City, SUM(r.Fare) AS Total_Revenue
```

```
FROM Rides r
JOIN Users u ON r.UserID = u.UserID
WHERE r.Status = 'Completed'
GROUP BY u.City
ORDER BY Total_Revenue DESC;
```

→ **Q4: Most used promo codes**

```
SELECT p.Code, COUNT(pu.UsageID) AS Times_Used
FROM Promos p
JOIN Promo_Usage pu ON p.PromoID = pu.PromoID
GROUP BY p.PromoID
ORDER BY Times_Used DESC
LIMIT 5;
```

→ **Q5: Rides where payment failed**

```
SELECT r.RideID, u.Name AS Rider, r.Fare, p.Payment_Mode
FROM Rides r
JOIN Users u ON r.UserID = u.UserID
JOIN Payments p ON r.RideID = p.RideID
WHERE p.Status = 'Failed';
```

Conclusion

Rapido's success as India's largest bike taxi platform is directly enabled by a clean, normalized data model that handles high-volume ride requests, multi-modal payment systems, and a dual-sided captain-rider marketplace. This schema captures all of that in 7 well-reasoned tables with clear foreign key relationships — making it both operationally sound and analytically powerful.

*Thank
you!*